

# LearnHub

## ONLINE LEARNING PLATFORM

---

System Design — Final Examination Project Documentation

**Submitted by:** Raj Koli

**Roll No:** 150096724145

**Topic:** Designing an Online Learning Platform like Coursera

**Course:** System Design

**GitHub Repository:** <https://github.com/Rajkoli145/LearnHub>

**Date:** June 2026

# 1. Problem Statement

---

Learners lack a single, structured platform to discover quality courses, enroll, follow an organized curriculum, and validate their understanding through assessments. Instructors lack a simple way to publish structured course content and measure learner performance. A scalable system is needed that cleanly separates concerns — authentication, course management, enrollment, and assessment — so each can evolve and scale independently.

# 2. Proposed Solution

---

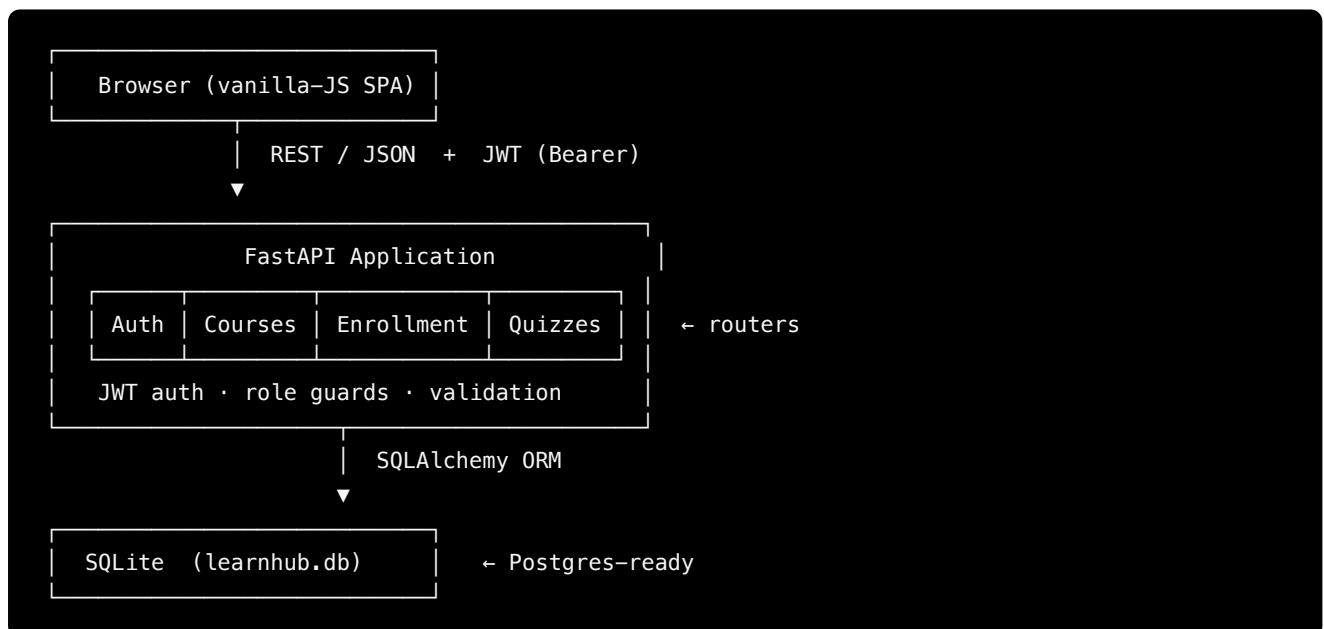
LearnHub is a modular web platform built on a **FastAPI** backend exposing a REST API, backed by a relational database via the **SQLAlchemy** ORM, with a lightweight JavaScript frontend. Functionality is split into four independent modules:

- **Auth** — JWT-based authentication with role-based access (learner, instructor, admin).
- **Courses** — instructors create courses organized into sections and lessons; learners browse, search, and filter.
- **Enrollment** — learners enroll in courses and track lesson-level progress.
- **Quizzes** — instructors author quizzes; the server auto-grades submissions and enforces attempt limits.

The modular router design mirrors a microservice-style separation while remaining a single deployable app — each module can later be extracted into its own service.

# 3. System Architecture

---



Request flow: the client sends JSON with a JWT in the `Authorization` header → FastAPI validates the token and the request body (Pydantic) → the matching router enforces role/ownership rules → SQLAlchemy reads/writes the database → a validated JSON response is returned. Detailed architecture, sequence, and ER diagrams are maintained in the `learnhubdiagrams/` folder.

## 4. Module Description

| Module     | Responsibility                                                                                     | Key Endpoints                                                                                  |
|------------|----------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| Auth       | Registration, login, JWT issuance, role enforcement, password hashing (PBKDF2).                    | POST /api/auth/register<br>POST /api/auth/login<br>GET /api/auth/me                            |
| Courses    | Course/section/lesson CRUD, public catalog with search & category filter, ownership checks.        | GET /api/courses<br>GET /api/courses/{id}<br>POST /api/courses                                 |
| Enrollment | Enroll learners, list "My Learning", update progress percentage.                                   | POST /api/enrollments/{cid}<br>GET /api/enrollments<br>PATCH .../progress                      |
| Quizzes    | Author quizzes/questions, serve quizzes without leaking answers, auto-grade, enforce max attempts. | POST<br>/api/courses/{id}/quizzes<br>GET /api/quizzes/{id}<br>POST<br>/api/quizzes/{id}/submit |

## 5. Database Design

Ten relational tables model the domain. Primary relationships:

- `users` 1—N `courses` (an instructor owns many courses)
- `courses` 1—N `sections` 1—N `lessons` 1—N `resources`
- `users` N—N `courses` via `enrollments`
- `courses` 1—N `quizzes` 1—N `questions`
- `quiz_attempts` 1—N `attempt_answers` (one row per answered question)

| Table           | Important Columns                                                                              |
|-----------------|------------------------------------------------------------------------------------------------|
| users           | id, email (unique), password_hash, full_name, role, is_verified, created_at                    |
| courses         | id, instructor_id (FK), title, description, category, price, is_free, rating_avg, review_count |
| sections        | id, course_id (FK), title, position                                                            |
| lessons         | id, section_id (FK), title, video_url, duration, is_free_preview, position                     |
| resources       | id, lesson_id (FK), title, url, type                                                           |
| enrollments     | id, user_id (FK), course_id (FK), progress, enrolled_at                                        |
| quizzes         | id, course_id (FK), title, pass_mark, max_attempts, order_index                                |
| questions       | id, quiz_id (FK), type, prompt, options (JSON), correct_answer, position                       |
| quiz_attempts   | id, quiz_id (FK), enrollment_id (FK), user_id (FK), score, passed, attempt_number              |
| attempt_answers | id, attempt_id (FK), question_id (FK), answer_given, is_correct                                |

## 6. Technology Stack

---

| Layer         | Technology             | Why                                                    |
|---------------|------------------------|--------------------------------------------------------|
| Language      | Python 3.11+           | Required; rich ecosystem                               |
| API framework | FastAPI                | Async, auto OpenAPI docs, Pydantic validation          |
| ORM           | SQLAlchemy 2.0         | DB-agnostic models; easy SQLite→Postgres               |
| Database      | SQLite                 | Zero-setup; file-based; production swaps to PostgreSQL |
| Auth          | PyJWT + hashlib PBKDF2 | Stateless tokens; no native build deps                 |
| Server        | Uvicorn (ASGI)         | Fast async server                                      |
| Frontend      | HTML/CSS/JS            | No build step; easy to run and demo                    |

## 7. Implementation Details

**Authentication.** Passwords are hashed with PBKDF2-HMAC-SHA256 (200k iterations, per-user salt) using the standard library. On login the server issues a JWT carrying the user id and a 24-hour expiry. Protected routes depend on `get_current_user`; role-restricted routes use a `require_role(...)` dependency factory.

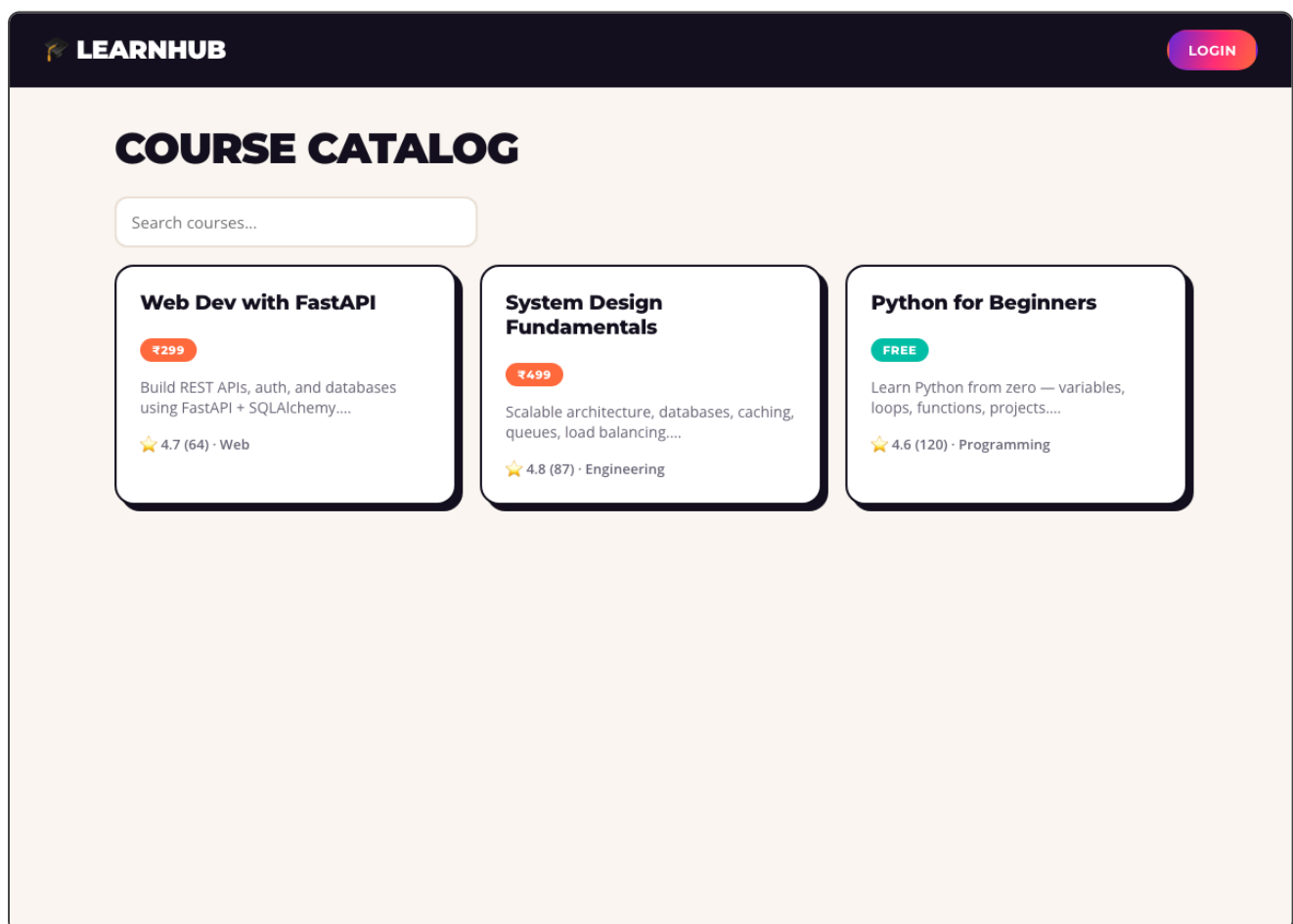
**Quiz auto-grading.** The quiz-fetch endpoint deliberately strips `correct_answer` so it can never be read from the browser. On submit, the server compares each answer case-insensitively, computes a percentage score, marks the attempt passed/failed against the quiz `pass_mark`, and persists every answer for review. `max_attempts` is enforced server-side.

**Validation & safety.** Every request/response is typed with Pydantic schemas, giving automatic 422 errors on bad input and a self-documenting OpenAPI spec. Ownership checks prevent one instructor from editing another's course.

```
POST /api/quizzes/1/submit          # all-correct submission
→ { "score": 100.0, "passed": true, "correct": 3, "total": 3, "attempt_number": 1 }

POST /api/courses (as learner)      # role guard blocks non-instructors
→ 403 { "detail": "Requires role: instructor, admin" }
```

## 8. Screenshots



Course catalog — search, ratings, free/paid badges.

## LOGIN

**LOGIN**

No account? [Register](#)

Demo: learner@ / instructor@ / admin@learnhub.dev ·  
Passw0rd!

Login screen with demo accounts.

# LearnHub API

1.0.0 OAS 3.1

/openapi.json

Online learning platform — System Design final project.

Authorize

## Auth

POST /api/auth/register Register

POST /api/auth/login Login

GET /api/auth/me Me

## Courses

GET /api/courses List Courses

POST /api/courses Create Course

GET /api/courses/{course\_id} Course Detail

DELETE /api/courses/{course\_id} Delete Course

POST /api/courses/{course\_id}/sections Add Section

POST /api/courses/sections/{section\_id}/lessons Add Lesson

## Enrollment

POST /api/enrollments/{course\_id} Enroll

Auto-generated Swagger API documentation at /docs.

Additional screens (My Learning dashboard, quiz attempt & result) can be captured after logging in locally with the demo accounts.

## 9. Future Scope

- Razorpay payment gateway with webhook-confirmed paid enrollment.
- Media uploads to S3/CDN with presigned URLs and video streaming.
- Migration to PostgreSQL with Alembic migrations; full-text search ranking.
- ML-based recommendation engine (scikit-learn) for a personalized feed.
- Celery task queue for emails, certificate generation, and analytics.
- Email/OTP verification, password reset, and instructor payout management.